Homework 5 Report — Ontology-based Semantic Grounding (Group 02)

Course: NYCU AI Capstone 2026 Spring **Group 02 members:** 彭程 (112550128), 周宇彥 (112550148), 張紹安 (112550166), 余逸翔 (112550027), 張盛瑋 (112550117), 張祐廷 (112550116)
Repository: https://github.com/VicsonPeng/semantic-affordance-grounding

1. Goal and division of labour

The goal is a small but semantically explicit ontology that lets a robot ground the objects it perceives in the course tasks and **infer** which are graspable. Work was split across the group (see `CONTRIBUTING.md`):

Role	Member	Deliverable
A	彭程	repo structure + ontology skeleton/metadata
B	周宇彥	`cap:GraspableObject` inference rule
C	張紹安	cup-stacking instances (primary task)
D	余逸翔	baseline instances (cutlery, toy blocks, basket)
E	張盛瑋	OWL reasoning + SPARQL + results
F	張祐廷	README + this report

2. Design rationale

A common, explicitly warned-against error is to call every task object "graspable." We avoid this by separating four layers — **object type**, **task role**, **affordance**, **instance** — and computing a fifth, **inferred** layer (`cap:GraspableObject`) with a reasoner. This makes the difference between *recognition* ("this is a plate") and *grounding* ("this plate is a placement reference the gripper will not pick up") explicit and queryable.

Each instance is connected to **explicit affordance individuals** via `cap:hasAffordance` (e.g. `g02:blueCupGraspingAffordance a cap:GraspingAffordance`). Modeling affordances as individuals grounds them as data and keeps the ontology inside OWL 2 RL, so graspability is derivable by a lightweight, fully reproducible Python reasoner as well as by Protégé + HermiT.

3. Namespace policy

- `cap:` = `https://hcis.io/ontology/aicapstone/2026/` — shared course vocabulary, reused directly; no new classes are placed here.

- `g02:` = `https://hcis.io/ontology/aicapstone/2026/group02/` — Group 02's modeling space: all individuals plus the two group schema terms.

The ontology carries the full metadata block (title, description, creator, creation date, license, preferred prefix and URI) and `owl:imports` the course ontology.

4. Reused vs. newly introduced terms

Reused from `cap:`: `PhysicalObject`, `RobotAgent`, `EndEffector`, `ManipulationTask`, the affordance classes, the role classes, the object classes (`Cup`, `Knife`, `Fork`, `Plate`, `ToyBlock`, `Basket`), and properties `hasAffordance`, `hasTaskRole`, `hasTargetObject`, `hasReferenceObject`, `canBeManipulatedBy`, `hasColor`, `hasObjectLabel`, `hasPoseFrame`, `hasApproxWidth`.

Reasoning target defined by the group (reserved `cap:` term): `cap:GraspableObject`, with the `owl:equivalentClass` axiom from spec Listing 5.

New under `g02:`: `g02:hasMaxGripWidth` (`owl:DatatypeProperty`, gripper capability), `g02:hasContainerTarget` (`owl:ObjectProperty`, links a task to its container), plus all task/object/affordance individuals, the robot and gripper.

5. Key axiom and reasoning pattern

```
cap:GraspableObject
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            cap:PhysicalObject
            [ a owl:Restriction ;
              owl:onProperty cap:hasAffordance ;
              owl:someValuesFrom cap:GraspingAffordance ]
        )
    ] .
```

DL: `GraspableObject ≡ PhysicalObject ⊓ ∃ hasAffordance.GraspingAffordance`.

The course ontology also constrains object classes with existential restrictions (e.g. `Cup ⊑ ∃hasAffordance.GraspingAffordance`). Those are superclass-side existentials, which are **outside** OWL 2 RL, so the reasoner will not invent an affordance for a bare `cap:Cup` instance — which is exactly why each instance is given an explicit affordance individual. OWL 2 RL rules that fire per grasp target:

Rule	Effect on `g02:blueCup01`
cax-sco	`a cap:Cup`, `Cup ⊑ PhysicalObject` ⇒ `a cap:PhysicalObject`
cls-svf1	`hasAffordance g02:blueCupGraspingAffordance` (a `cap:GraspingAffordance`) ⇒ member of the existential restriction class
cls-int1	in both conjuncts ⇒ member of the intersection class
cax-eqc1	intersection `owl:equivalentClass cap:GraspableObject` ⇒ `a cap:GraspableObject`

It does **not** fire for the plate (support affordance) or basket (containment affordance), because cls-svf1 needs a `cap:GraspingAffordance` value.

6. Query results

`queries/graspable_objects.rq` returns exactly the six inferred grasp targets:

```
blueCup01, fork01, knife01, pinkCup01, toyBlock01, toyBlock02   (6 rows)
```

The same query over the **asserted-only** graph returns **0 rows**, confirming the membership is inferred, not asserted. `queries/task_objects.rq` lists every task object and shows the plate and basket with an empty `graspable` column. Full output is under `results/`. This matches the spec's expectation that the cups, knife, fork, and blocks are graspable, while plate/basket inclusion depends on modeling — here, by design, they are excluded.

7. Design choices

1. **Affordances as individuals** — grounds the affordance layer and keeps the ontology in OWL 2 RL, so results reproduce with a free Python toolchain.
2. **Plate and basket are not graspable** — they keep their support/containment affordances and task roles but no grasping affordance.
3. **`cap:GraspableObject` defined in the group file** — respects the namespace policy (reserved `cap:` term) while supplying the missing defining axiom.
4. **Cleaned inferred export** — trivially-true closure triples are stripped so the meaningful entailments are readable.

8. Limitations and possible extensions

- **OWL 2 RL, not full OWL DL.** The reasoner does not do superclass-side existential reasoning, so a bare `cap:Cup` with no explicit affordance would not be classified graspable; we sidestep this by asserting affordance individuals. Protégé + HermiT would handle that pattern directly.
- **Gripper capability declared but not enforced.** `g02:hasMaxGripWidth` and `cap:hasApproxWidth` are present; a future SHACL shape or rule could mark an object

graspable only when its width fits the gripper (spec §15/§19).
- **Pose data as identifiers.** `cap:hasPoseFrame` stores frame strings for traceability to the simulation, not full transforms.

9. Conclusion

The repository demonstrates a complete semantic grounding loop: perceived object → class definition → affordance representation → OWL reasoning pattern → inferred graspability → queryable SPARQL result. Graspability is derived by a transparent, reproducible step, and the deliberate exclusion of the plate and basket shows the model captures *why* an object is or is not graspable, not merely that it is task-relevant.